

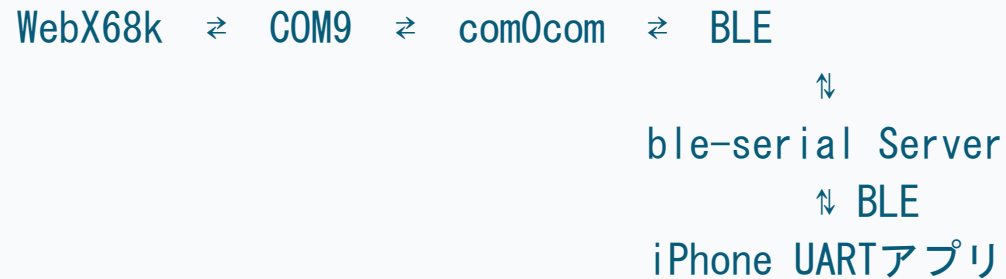
Windows / BLEシリアル 最小セットアップ

WebX68k・com0com・ble-serial・iPhone

WebX68k は COM9 を開き、ble-serial は対向の BLE を開きます。

この構成では、iPhoneからWindows PCを探して接続します。

BLEでは、接続を開始するiPhoneをCentral、接続を待つWindows PCをPeripheralと呼びます。



BLE advertising：iPhoneから検索できるよう、
機器名やサービス情報を周囲へ発信する処理です。

Python環境と必要なもの

Python 3.11 (Windows x64) と専用venvを1組使います。
Bluetoothアダプターには、Windows上でのBLE Peripheral対応が必要です。

Python	3.11.9
ble-serial	3.0.0
bless	0.2.6 + 次頁以降の回避修正
bleak	0.22.3
bleak-winrt	1.2.0
pysetupdi	GitHubから導入

pyserial・pywin32等はpipが依存関係として導入します。

com0comの COM9 と BLE

com0com はセットアップ済とします。

WebX68kとble-serialの接続を終了し、管理者PowerShellで操作します。

まず既存ペアを一覧表示します。COM9 / BLEの重複作成は避けます。

```
Set-Location "C:\Program Files (x86)\com0com"  
.\%setupc.exe list
```

どちらの名前も未使用の場合だけ、新しいペアを作成します。

```
.\%setupc.exe install PortName=COM9 PortName=BLE
```

既存が「CNCA1 PortName=COM2 / CNCB1 PortName=BLE」の場合の変更例。

CNCA1は例です。実際のlist結果にあるCOM2側の識別子を使います。

```
.\%setupc.exe change CNCA1 PortName=COM9  
.\%setupc.exe list
```

COM9が他の機器に割り当て済みの場合は変更を中止します。

既存のcom0comドライバーを利用します。再インストールは不要です。

Python 3.11の専用venv

以下は通常のPowerShellで実行します。Python 3.11が既にあれば利用します。

```
py -3.11 --version
```

Python 3.11がない場合だけインストールし、PowerShellを開き直します。

```
winget install -e --id Python.Python.3.11
```

専用venvを新規に作る場合のコマンド。既存の311環境は再作成不要です。

```
py -3.11 -m venv "$env:USERPROFILE¥venvs¥ble-serial311"  
$blePython = "$env:USERPROFILE¥venvs¥ble-serial311¥Scripts¥python.exe"
```

PowerShellを開き直すたびに、最後の変数代入を実行します。
変数で指定した実行ファイルを呼び出す際は、先頭に & が必要です。

必要なライブラリの導入

この組み合わせを同じvenvへ導入します。--userは付けません。
--no-cache-dirで新しいpipキャッシュの蓄積を抑えます。

```
& $blePython -m pip install --no-cache-dir `
    "bleak-winrt==1.2.0" "bleak==0.22.3" `
    "bless==0.2.6" "ble-serial==3.0.0"
```

Windowsの追加依存はGitHubから導入します。

```
$setupdiUrl = "https://github.com/gwangyi/pysetupdi/archive/refs/heads/master.zip"
& $blePython -m pip install --no-cache-dir $setupdiUrl
```

行末の`はPowerShellの継続記号です。後ろに空白を入れません。
[server]を省き、必要なblessとpysetupdiを明示的に導入しています。
導入後にblessを再インストールすると、次の回避修正も再適用が必要です。

CreateFileエラーの回避修正

bless 0.2.6は名前変更を行わない場合にもアダプターを直接開きます。
CreateFileエラーを避けるため、名前変更時だけ開くように2か所を修正します。

```
$serverPy = "$env:USERPROFILE¥venvs¥ble-serial311¥Lib¥site-packages"  
$serverPy += "¥bless¥backends¥winrt¥server.py"  
Copy-Item -LiteralPath $serverPy -Destination "$serverPy.before-fix.bak"  
notepad $serverPy
```

__init__内の「self._adapter: BLEAdapter = BLEAdapter()」を置き換えます。

```
self._adapter: Optional[BLEAdapter] = (  
    BLEAdapter() if name_overwrite else None  
)
```

start内のifを下記へ変更し、UTF-8で保存します。既に同じなら修正不要です。

```
if self._name_overwrite and self._adapter is not None:  
    self._adapter.set_local_name(self.name)
```

起動前の確認とServer起動

依存関係とServerオブジェクトの生成を確認します。

```
& $blePython -m pip check
```

```
& $blePython -c "import bleak_winrt; from bless import BlessServer; BlessServer(name='WebX68k'); print('OK')"
```

No broken requirements found. と OK を確認してから起動します。

\$nusは、Nordic UART Service (NUS) のサービスUUIDを入れる変数です。

UUIDはサービスを識別する番号で、NUSはBLE上でシリアル通信を行うための仕組みです。

以下の値をそのまま使い、-s \$nusでble-serialに指定します。

```
$nus = "6e400001-b5a3-f393-e0a9-e50e24dcca9e"
```

```
& $blePython -m ble_serial -g server -n WebX68k `
    -p BLE -s $nus -v
```

Server startup successful等の起動ログを確認します。停止はCtrl+Cです。

Windows PCがBLE接続先の情報を発信し、iPhoneから検索できる状態になります。

iPhoneと CTTY AUX の操作

1. WebX68kのシリアル接続でCOM9を選びます。
X68000側とブラウザー側のボーレート設定を合わせます。
2. iPhoneのBluefruit ConnectでPCを探し、ConnectからUARTを開きます。
表示名はWebX68kではなくWindows PC名になる場合があります。
3. X68000のコマンドラインで CTTY AUX を実行します。
以後、コマンド入力と応答はシリアル端末側で扱います。
4. iPhoneから DIR とCR (0x0D) を送信します。
文字だけ届いて実行しない場合は改行コードを確認します。
5. iPhoneから CTTY CON とCRを送ると、画面側の入力へ戻ります。

Bluefruit Connectの改行はCRと限りません。CRを送れない場合は、nRF ConnectのNUS RXへ 44 49 52 0D (DIR + CR) を書いて切り分けます。最初はASCIIで確認します。日本語にはShift_JIS対応または変換が必要です。

参照資料

ble-serial : Windows設定・Serverモード

<https://github.com/Jakeler/ble-serial>

bleak-winrt : Python別のWindows配布バイナリ

<https://pypi.org/project/bleak-winrt/#files>

bless : CreateFile問題と今回の回避修正の根拠

<https://github.com/kevincar/bless/issues/148>

<https://github.com/kevincar/bless/blob/v0.2.6/bless/backends/winrt/server.py>

com0com : 設定コマンド

<https://com0com.sourceforge.net/doc/UsingCom0com.pdf>

Bluefruit Connect : UART Terminal

<https://learn.adafruit.com/bluefruit-le-connect/uart-terminal>

Nordic UART Service : サービスUUIDと送受信の仕組み

https://docs.nordicsemi.com/r/bundle/nrf5_sdk_v15.0.0/page/ble_sdk_app_nus_eval.html